SAVEETHA SCHOOL OF ENGINEERING
C PROGRAMMING ASSIGNMENT – Logic Analysis and Design Challenges

Field	Entry
Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Logic Analysis and Design Challenges (Q1–Q7, 50 marks)
GitHub Repository URL	https://github.com/bavadarini28 /CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

Q2 – When Does Short-Circuit Evaluation Matter?

Experiment design

1,000 passengers generated with srand(42): about 30% senior, 50% eligible class, 5% hold a card. slow(i) increments a global call counter. Four formulations of the same rule were run on the same data.

Formulation	Calls to slow()	Granted	Why
(S && E) || slow() [cheap first]	836	211	slow() runs only when (S && E) is false: 1000 − 164
slow() || (S && E) [slow first]	1000	211	slow() always runs first
(S && E) | slow() [non-short-circuit]	1000	211	| evaluates both sides always
Nested if / else if (cheap first)	836	211	else-branch only reached when the first test fails

Operand not evaluated: in a && b, b is skipped when a is false; in a || b, b is skipped when a is true. Test: x=0; (0 && ++x) leaves x = 0, (1 || ++x) leaves x = 0; but with & and | the ++x always runs and x becomes 1.

& versus &&: & is a bitwise operator: both operands are always evaluated, with no guaranteed order, and on arbitrary integers (e.g. 2 & 1 = 0 although both are "true"). && yields 0 or 1 and short-circuits with a sequence point after the first operand.

Rule: for &&, put first the cheapest condition that is most likely false; for ||, the cheapest condition most likely true. Order is irrelevant when both operands are cheap and free of side effects (the saving is a few cycles), when the first operand is always decisive/never decisive, or when every operand must be evaluated anyway. Order is also functionally important when the first test guards the second (p != NULL && p->x).

Changed condition – audit log

Measured: with the cheap-first order only 836 of the 1000 passengers were logged (164 were never verified or recorded); with slow-first all 1000 were logged. The function now has a side effect that must happen for every passenger, so short-circuit skipping is a correctness bug, not an optimisation. Recommendation: call the verifier unconditionally and store the result (int card = slow_log(i);), then evaluate (S && E) || card. The

number of calls is then n regardless of order, so the ordering optimisation no longer applies. If only some passengers must be audited, keep the cheap-first order and document that unaudited cases are intentional.

Alternative Concept and Complexity Analysis

Nested if statements behave like && / ||: the slow function is called only when the cheap test fails – expected calls $n(1 - p)$ where $p = P(S \&\& E)$ (836 here), worst case n, so O(n) with a smaller constant. With & and | the slow function is always called: exactly n calls (1000), plus no protection for guarded expressions. Space is O(1) for all three.